CHARUSAT®
CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF TECHNOLOGY AND ENGINEERING
DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH

DEPARTMENT OF INFORMATION TECHNOLOGY

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms

DEPSTAR

Practical 6: Stack Implementation using Array and Linked List, with Expression Processing

Practical Aim:

To implement stack using array and linked list, and apply stack-based logic to expression conversion, evaluation, and bracket validation problems.

Problem 1 :

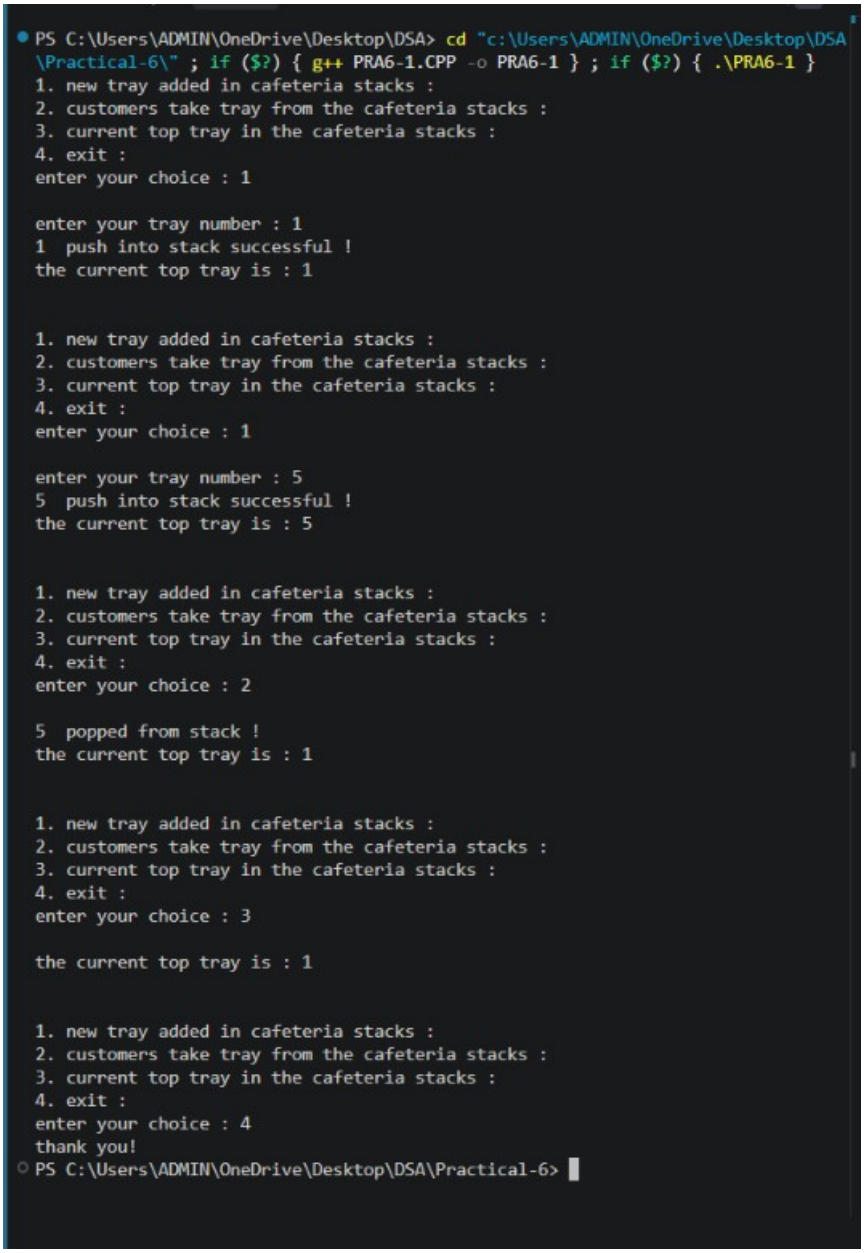
A cafeteria stacks clean trays on a fixed-size counter. New trays are always placed on top, and customers always take from the top. The counter can hold at most n trays at a time — if it is full, no more trays can be added, and if it is empty, no tray can be taken. Given a sequence of place and take operations, implement this fixed-capacity tray stack and print the current top tray after each operation. Report an error if a place or take operation cannot be performed.

Describe how your solution tracks whether the stack is full or empty. What are the exact conditions you check before a push and before a pop, and what does your program do when those conditions fail?

For Code:

https://github.com/ayushposhiya00-lab/FDSA/tree/main/DSA/Practical-6

Output:


PS C:\Users\ADMIN\OneDrive\Desktop\DSA> cd "c:\Users\ADMIN\OneDrive\Desktop\DSA\Practical-6\" ; if ($?) { g++ PRA6-1.CPP -o PRA6-1 } ; if ($?) { .\PRA6-1 }
1. new tray added in cafeteria stacks :
2. customers take tray from the cafeteria stacks :
3. current top tray in the cafeteria stacks :
4. exit :
enter your choice : 1
enter your tray number : 1
1 push into stack successful !
the current top tray is : 1
1. new tray added in cafeteria stacks :
2. customers take tray from the cafeteria stacks :
3. current top tray in the cafeteria stacks :
4. exit :
enter your choice : 1
enter your tray number : 5
5 push into stack successful !
the current top tray is : 5
1. new tray added in cafeteria stacks :
2. customers take tray from the cafeteria stacks :
3. current top tray in the cafeteria stacks :
4. exit :
enter your choice : 2
5 popped from stack !
the current top tray is : 1
1. new tray added in cafeteria stacks :
2. customers take tray from the cafeteria stacks :
3. current top tray in the cafeteria stacks :
4. exit :
enter your choice : 3
the current top tray is : 1
1. new tray added in cafeteria stacks :
2. customers take tray from the cafeteria stacks :
3. current top tray in the cafeteria stacks :
4. exit :
enter your choice : 4
thank you!
PS C:\Users\ADMIN\OneDrive\Desktop\DSA\Practical-6>

Problem Faced:

The main challenge was deciding the exact conditions for overflow and underflow and making sure they are checked before the top index is changed. With a fixed-size array, an incorrect check (for example top == n instead of top == n - 1) would write outside the array or read an invalid element. It was also necessary to display the correct top tray, or a proper message, when the stack becomes empty.

Outcome/Conclusion:

A fixed-capacity tray stack was implemented using an array of size n with an index called top that is initialised to -1. The stack is empty when top == -1 and full when top == n - 1.

Before a push (place), the program checks top == n - 1; if true, it reports “Stack Overflow” and leaves the stack unchanged, otherwise it increments top and stores the tray. Before a pop (take), the program checks top == -1; if true, it reports “Stack Underflow”, otherwise it removes the tray at top and decrements top. After every valid operation the current top tray is printed (or a message that the stack is empty).

Problem 2 :

A web browser keeps track of pages visited so that the back button always returns to the most recently visited page. Unlike a fixed counter, the browser has no hard limit on how many pages it can remember — it grows as the user visits more pages and shrinks as they press back. Given a sequence of visit and back operations, implement this unlimited page history and print the current page after each operation.

Describe how your solution differs from Problem 1 in how it handles memory. What happens in your solution when the user presses back on the very first page with no history left?

For Code:

https://github.com/ayushposhiya00-lab/FDSA/tree/main/DSA/Practical-6

Output:



Problem Faced:

The main challenge was handling dynamic memory correctly. A new node has to be created for every visit and freed on every back operation, and the head pointer must always point to the current page. The special case of pressing back when there is no earlier page also had to be handled so that the history is never left empty by mistake or accessed through a NULL pointer.

Outcome/Conclusion:

The unlimited page history was implemented as a stack using a singly linked list, where the head node is the current page. A visit operation creates a new node and inserts it at the head; a back operation removes the head node, frees its memory, and the next node becomes the current page. The current page is printed after every operation.

The difference from Problem 1 is memory handling: the array stack reserves a fixed amount of memory in advance and can overflow, whereas the linked list stack allocates one node at a time only when needed and releases it when it is no longer needed, so there is no fixed capacity check. When the user presses back on the very first page, there is no earlier page to return to, so the program reports that no history is available and keeps the current page unchanged instead of accessing a NULL pointer.

Problem 3 :

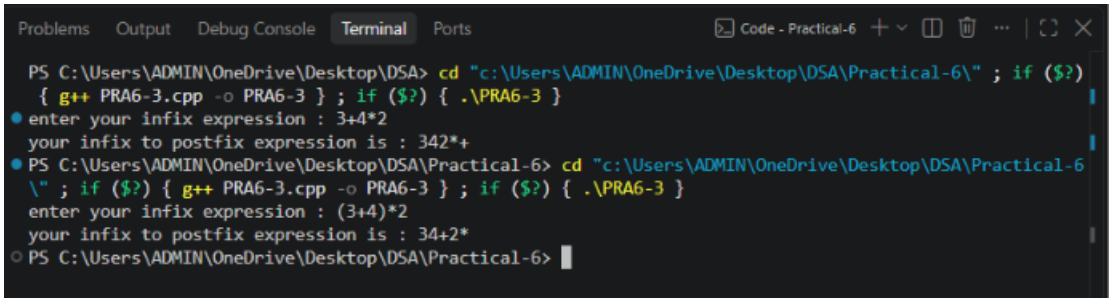
A calculator application receives arithmetic expressions typed by users in the usual way — numbers and operators written in between, with brackets to control priority (e.g. 3 + 4 * 2 or (3 + 4) * 2). Internally the calculator needs to convert each expression into a form where operators appear after their operands, so it can evaluate them without needing to re-read brackets. Given an infix expression, convert it to the equivalent postfix form and print the result.

Describe the role the stack plays in your conversion. What does your solution do when it encounters a closing bracket, and what happens to the operators that were waiting on the stack at that point?

For Code:

https://github.com/hetbhimani5/FDSA-Batch-A-ARP/tree/main/Practical-6

Output:



Problem Faced:

The main challenge was applying operator precedence and bracket handling correctly during the conversion. Deciding when a waiting operator on the stack must be moved to the output (based on the precedence of the incoming operator), and making sure the opening bracket is removed without being printed, required careful tracing of the stack after each character.

Outcome/Conclusion:

The infix to postfix conversion was implemented using a stack that holds the operators (and opening brackets) which are still waiting for their operands. The

expression is scanned from left to right: operands are added directly to the output; an opening bracket is pushed; for an operator, all operators on the stack with higher or equal precedence are popped to the output first and then the new operator is pushed; at the end, all remaining operators are popped to the output.

When a closing bracket is encountered, all operators waiting on the stack are popped and added to the output one by one until the matching opening bracket is found; that opening bracket is then popped and discarded, so brackets never appear in the postfix output. For example, 3 + 4 * 2 becomes 3 4 2 * +, and (3 + 4) * 2 becomes 3 4 + 2 *.

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation:

[Problem 1] A circular array is one way to reuse freed slots without shifting all elements. Without implementing it, explain what change would need to be made to your front and rear pointer logic to make the array behave circularly, and what new condition would indicate a full array.

Sol: To make the array circular, the front and rear indices must not stop at the end of the array but wrap around to index 0 using the modulo operator, i.e. rear = (rear + 1) % n and front = (front + 1) % n. In this way, slots that were freed at the beginning of the array can be reused by later insertions without shifting any elements. Because front == rear can then mean either empty or full, the full condition must be changed: the array is full when (rear + 1) % n == front (one slot is kept unused), or when a separate count of stored elements equals n. The empty condition remains front == rear (or count == 0).

[Problem 2] Your linked list stack has no fixed capacity limit. But in practice, can it still run out of space? What determines the actual limit, and why is that limit invisible in your code?

Sol: Yes, it can still run out of space. Every node is allocated from the heap, and the heap is limited by the physical memory available and by the limits set by the operating system for the process. If too many pages are pushed, the allocation (new / malloc) eventually fails, by returning NULL or throwing an exception. This limit is invisible in the code because it is not a constant or a condition written in the program: the code simply requests a new node on every visit, and

the limit depends on the machine and the runtime state, so it only appears when an allocation fails (which is why the allocation result should be checked in a robust program).

[Problem 3] Operator precedence determines the order in which operators are placed into the output. Without tracing any expression, explain what would go wrong in your output if your solution treated all operators as having equal precedence.

Sol: If all operators had equal precedence, an operator already waiting on the stack would always be moved to the output before the next operator is pushed, so the postfix output would simply follow the left-to-right order of the expression. Operators such as * and / would no longer be placed ahead of + and -, and the postfix expression would represent a different order of evaluation. For example, 3 + 4 * 2 would be converted to 3 4 + 2 * (result 14) instead of the correct 3 4 2 * + (result 11). Precedence is therefore what keeps lower-priority operators waiting on the stack until the higher-priority operations have been output, and ignoring it gives wrong results even though the output looks like a valid postfix expression.